Bayes Filter and Kalman Filter

Preksha Samajpati
Department of Computer Science and Engineering
Manipal Institute of Technology, MAHE
Manipal, India
preksha.mitmpl2026@learner.manipal.edu

Abstract—A driverless car never observes its true state: its sensors are noisy and its motion model is imperfect. Recursive filters fuse the two into a single best estimate together with a measure of its uncertainty. This report presents the general Bayes filter, identifies the assumptions under which it reduces to the Kalman filter, walks through the Kalman filter equations step by step, and explains where such a filter sits in the driverless (DV) software pipeline of a Formula Student car.

Keywords—Bayes filter, Kalman filter, state estimation, sensor fusion, autonomous racing

I. Introduction

A driverless race car must decide where to steer using a state it can never observe directly. Its position, heading and velocity are only inferred from sensors such as LiDAR, GNSS, an IMU and wheel-speed encoders, and each of these is noisy, drifts or updates at a different rate. The car's motion model is imperfect too, since wheel slip and unmodelled dynamics all push the real motion away from what the equations predict. Trusting either source alone leads to errors that propagate straight into planning and control, which at racing speeds can mean a missed cone or a lost run.

Recursive Bayesian filtering addresses this by treating the state as a probability distribution rather than a single value. At each time step the filter predicts the new state from a motion model, then corrects that prediction with the latest measurement, weighting each by how uncertain it is . The Kalman filter, introduced in 1960, is the best-known special case. Under linear models and Gaussian noise it gives this update in closed form and is optimal in the minimum-variance sense.

This report develops the idea from the general to the specific. Section II presents the Bayes filter and defines its terms. Section III states the assumptions that reduce it to the Kalman filter and what fails without them. Section IV derives the Kalman filter equations and the role of the Kalman gain. Section V explains where the filter fits in the driverless (DV) pipeline of our car.

II. The Bayes Filter

A. Belief and Markov Assumptions

Let $\mathbf{x}_t$ be the hidden state of the car at time t (for example position, heading and velocity), $\mathbf{u}_t$ the control input (steering, throttle) and $\mathbf{z}_t$ the noisy measurement. The quantity the filter maintains is the *belief*

$$\mathrm{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}). \tag{1}$$

Two Markov assumptions make this recursive: (i) the state is *complete*, so given $\mathbf{x}_{t-1}$ and $\mathbf{u}_t$ the past carries no extra information about $\mathbf{x}_t$; (ii) given $\mathbf{x}_t$, the measurement $\mathbf{z}_t$ is independent of everything else. The filter then alternates two steps (Fig. 1) [1], [2].

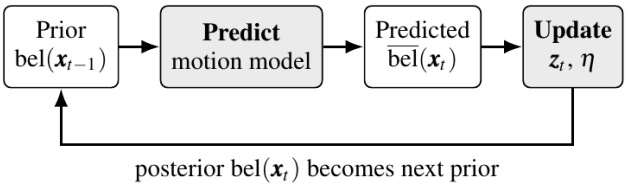


Fig. 1. The recursive Bayes filter loop. One predict–update cycle runs per time step.

B. Step 1: Prediction (Time Update)

Using the law of total probability and the Markov property,

$$\overline{\mathrm{bel}}(\mathbf{x}_t) = \int p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)\, \mathrm{bel}(\mathbf{x}_{t-1})\, d\mathbf{x}_{t-1}. \tag{2}$$

Here $\mathrm{bel}(\mathbf{x}_{t-1})$ is the *prior belief*, everything known before the new data arrives. $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ is the *motion (transition) model*: the probability of ending in $\mathbf{x}_t$ if the car started in $\mathbf{x}_{t-1}$ and was commanded $\mathbf{u}_t$. The integral averages the motion model over every state the car might have been in, weighted by how plausible each was. The result $\overline{\mathrm{bel}}(\mathbf{x}_t)$ is the *predicted belief*; it is at least as uncertain as the prior because the motion model adds noise.

C. Step 2: Measurement Update (Correction)

Applying Bayes' rule to the new measurement,

$$\mathrm{bel}(\mathbf{x}_t) = \eta\ p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{\mathrm{bel}}(\mathbf{x}_t). \tag{3}$$

$p(\mathbf{z}_t \mid \mathbf{x}_t)$ is the *measurement likelihood (sensor model)*: how probable the reading $\mathbf{z}_t$ would be if the true state were $\mathbf{x}_t$. Multiplying it with the predicted belief up-weights states that explain the reading and down-weights those that do not. The *normaliser* is

$$\eta = [\int p(\mathbf{z}_t \mid \mathbf{x}_t)\, \overline{\mathrm{bel}}(\mathbf{x}_t)\, d\mathbf{x}_t]^{-1}. \tag{4}$$

It is a constant over $\mathbf{x}_t$ that forces the posterior to integrate to one, and it is the reciprocal of the *evidence* $p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t})$. Together, (2) and (3) form the complete Bayes filter. It is exact, but in general the integrals have no closed form and the belief cannot be stored with a finite number of parameters.

III. From Bayes to Kalman: Assumptions

The Kalman filter [3] is the Bayes filter under three assumptions:

- **Linear motion model:** $\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t + \mathbf{w}_t$.
- **Linear measurement model:** $\mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t$.
- **Gaussian noise and initial belief:** $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ and $\text{bel}(\mathbf{x}_0) = \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$, with noise white (uncorrelated in time).

Why this is tractable. A Gaussian is fully described by its mean and covariance. A linear map of a Gaussian is again Gaussian, and the sum of independent Gaussians is Gaussian, so the prediction integral (2) stays Gaussian with a closed-form mean and covariance. The product of two Gaussians (likelihood times predicted belief) in (3) is again an unnormalised Gaussian, and η follows analytically. The belief therefore remains $\mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$ at every step: the update of an entire function collapses into matrix algebra on $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, costing roughly $O(n^3)$ per step for an n-dimensional state. With these assumptions the Kalman filter is also the optimal minimum-variance estimator.

What goes wrong otherwise. If the motion function $f(\cdot)$ or measurement function $h(\cdot)$ is nonlinear (a car's heading-dependent kinematics, a range–bearing sensor), a Gaussian pushed through it is no longer Gaussian. The belief may become skewed or multimodal and the integrals have no closed form. Applying the Kalman equations anyway gives *approximate* answers whose errors can accumulate. The usual remedies are the Extended Kalman Filter (EKF, first-order linearisation with Jacobians, valid only when the nonlinearity is mild over the uncertainty), the Unscented Kalman Filter (UKF, propagating sigma points) and particle filters (sampling, no Gaussian assumption but costly). Heavy-tailed noise or outliers (for example GNSS multipath) violate the Gaussian assumption and can pull the estimate far off unless gated. Badly tuned $\mathbf{Q}$ and $\mathbf{R}$ make the filter over- or under-confident, and an overconfident filter can reject good measurements and diverge.

IV. The Kalman Filter in Detail

Let $\boldsymbol{\mu}_t$ and $\boldsymbol{\Sigma}_t$ be the posterior mean and covariance. The filter repeats the following cycle [1], [4].

A. Predict

$$\bar{\boldsymbol{\mu}}_t = \mathbf{A}\boldsymbol{\mu}_{t-1} + \mathbf{B}\mathbf{u}_t, \tag{5}$$

$$\bar{\boldsymbol{\Sigma}}_t = \mathbf{A}\boldsymbol{\Sigma}_{t-1}\mathbf{A}^\top + \mathbf{Q}. \tag{6}$$

Eq. (5) pushes the previous mean through the motion model to predict where the car is now. Eq. (6) propagates the uncertainty: $\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$ is how the old uncertainty is stretched or rotated by the dynamics, and $\mathbf{Q}$ is the new uncertainty injected by unmodelled effects (wheel slip, road bumps). Uncertainty only grows during prediction.

B. Update

$$\mathbf{S}_t = \mathbf{H}\bar{\boldsymbol{\Sigma}}_t\mathbf{H}^\top + \mathbf{R}, \tag{7}$$

$$\mathbf{K}_t = \bar{\boldsymbol{\Sigma}}_t\mathbf{H}^\top\mathbf{S}_t^{-1}, \tag{8}$$

$$\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + \mathbf{K}_t\,(\mathbf{z}_t - \mathbf{H}\bar{\boldsymbol{\mu}}_t), \tag{9}$$

$$\boldsymbol{\Sigma}_t = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\bar{\boldsymbol{\Sigma}}_t. \tag{10}$$

The *innovation* $\mathbf{y}_t = \mathbf{z}_t - \mathbf{H}\bar{\boldsymbol{\mu}}_t$ is the gap between the real measurement and what the prediction expected the sensor to read. $\mathbf{S}_t$ in (7) is its covariance: the predicted-state uncertainty seen through the sensor plus the sensor's own noise. The Kalman gain (8) decides how much of the innovation to believe. Eq. (9) corrects the prediction by $\mathbf{K}_t\mathbf{y}_t$, and (10) shrinks the covariance because information was added, so uncertainty falls during update. In practice the Joseph form, $(\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\boldsymbol{\Sigma}}(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top$, is preferred for numerical stability.

C. Meaning of the Kalman Gain

For a scalar state with $H = 1$ the gain reduces to

$$K_t = \frac{\bar{\sigma}_t^2}{\bar{\sigma}_t^2 + r}, \qquad \mu_t = (1 - K_t)\,\bar{\mu}_t + K_t\,z_t, \tag{11}$$

so the new estimate is a weighted average of prediction and measurement, weighted by their certainties.

- **Very noisy sensor** ($r \to \infty$): $K_t \to 0$. The measurement is almost ignored, the estimate follows the motion model, and $\sigma_t^2 \approx \bar{\sigma}_t^2$ (barely any uncertainty reduction).
- **Very accurate sensor** ($r \to 0$): $K_t \to 1$ (in general $\mathbf{K} \to \mathbf{H}^{-1}$). The estimate jumps to the measurement and $\sigma_t^2 \to 0$.

The filter therefore adapts automatically: it trusts whichever source, model or sensor, currently has the smaller variance. A large $\bar{\boldsymbol{\Sigma}}_t$ (a long time since a good fix) also raises the gain, so the filter leans on the next measurement more heavily.

V. Relevance to Our Driverless Car

A Formula Student DV stack is commonly organised as perception, state estimation and mapping, planning, and control (Fig. 2). The Kalman filter family appears at several points.

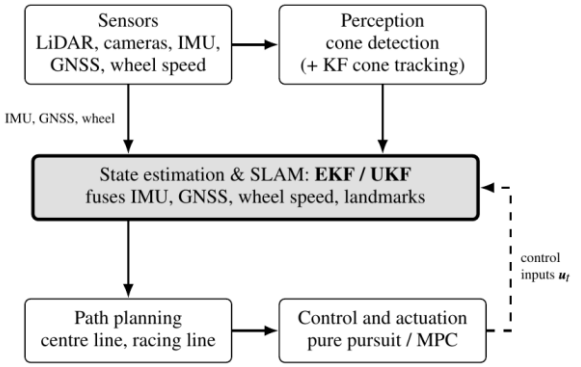


Fig. 2. Simplified DV pipeline. The shaded block is the main home of the Kalman filter; the dashed arrow feeds commands back as the control input.

A. Vehicle State Estimation (Main Use)

The car's pose and velocity, $\mathbf{x} = [x,\, y,\, \psi,\, v_x,\, v_y,\, \dot{\psi}]^\top$, are needed by planning and control but are not measured directly. The motion model (a kinematic or dynamic bicycle model driven by steering and throttle commands, or IMU integration) provides the predict step at high rate (IMU, 100 Hz or more). Wheel-speed sensors, GNSS (slower, absolute but noisy) and, in SLAM, LiDAR landmarks supply update

steps at their own rates. Because a Kalman filter simply skips the update when no measurement arrives, asynchronous multi-rate fusion is natural: IMU drift is bounded by GNSS and wheel speed, while GNSS jitter is smoothed by the IMU. Vehicle kinematics are nonlinear (heading enters through sine and cosine), so an EKF or UKF is used in practice. The Kalman gain then provides the automatic weighting discussed above, for example trusting wheel speed less during wheel slip if **R** is inflated.

B. Cone Tracking and SLAM

In perception, each detected cone can be given a small constant-position Kalman filter that averages repeated LiDAR and camera detections over frames, reducing jitter and false positives. Innovation gating decides whether a detection belongs to an existing track (data association). In unknown-track events, EKF-SLAM jointly estimates the car pose and cone positions in one state vector, with the full covariance capturing correlations between them (graph-based or particle-based SLAM is an alternative). In both cases the covariance output is useful downstream: planning can treat uncertain cones conservatively and control can be made aware of localisation confidence.

C. Practical Considerations

The filter needs sensor time-synchronisation, calibrated extrinsics and carefully tuned **Q** and **R** taken from sensor datasheets and logged data. Monitoring the innovation (normalised innovation squared) detects sensor faults and filter inconsistency, which matters for the safety of a driverless vehicle.

VI. CONCLUSION

The Bayes filter provides the general recipe: predict with the motion model, correct with the measurement likelihood, and normalise. Linear dynamics, Gaussian noise and a Gaussian initial belief turn that recipe into the closed-form, optimal Kalman filter, whose gain balances model against sensor by their relative uncertainties. Real vehicles violate linearity, so EKF and UKF variants form the core of state estimation in the DV pipeline.

VII. ACKNOWLEDGEMENT

I would like to thank the Driverless Subsystem seniors for providing the guidance and resources that helped me complete the task with a deeper understanding of Bayes and Kalman filter and the concepts discussed in this report.

VIII. REFERENCES

[1] J. W. Lambert, “Bayes filter.” [Online]. Available: https://johnwlambert.github.io/bayes-filter/

[2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[3] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[4] A. Becker, “Online Kalman filter tutorial,” KalmanFilter.NET. [Online]. Available: https://kalmanfilter.net/